

CSE422: Artificial Intelligence
Project Report

Machine Learning Techniques for Age Prediction: A Study on the Bank Marketing Dataset

Summer 2025
Section: 31
Group: 3

Members:

Name	ID
Rahabar Islam	23101147
Md Asif Ahemed	23101198

Introduction:..... 2
Dataset Description:.....2
Imbalanced Dataset:.....6
EDA:..... 7
Data Pre-Processing:.....10
Dataset Splitting:.....12
Model Training & Testing:.....13
Supervised:.....13
Unsupervised:..... 14
Comparison Analysis:.....16
Conclusion:.....17

Introduction:

In the banking sector, customer-related data provides valuable insights into financial behavior and marketing outcomes. The **Bank Marketing** dataset used in this project contains information about customers' personal attributes, contact details, and responses to previous campaigns.

This project reframes the dataset into a Regression task by predicting the **"Age"** of customers using available features. To achieve this, we designed a structured pipeline that included exploration of the dataset, correlation analysis, encoding, scaling, feature engineering, and model training. Data was carefully preprocessed through Label Encoding, One-Hot Encoding, and Log Transformation to handle categorical variables, binary features, and skewed distributions, and for numerical stability, Robust Scaling is used for further improvement.

Multiple models were implemented and compared, including Linear Regression, Decision Tree Regressor, and a Neural Network (MLP Regressor). Their performance was evaluated with metrics such as Mean Squared Error (MSE) and R^2 Score, highlighting differences in prediction accuracy. Additionally, an Unsupervised Learning approach was explored with K-Means Clustering, which grouped customers into meaningful age-based clusters: Young, Middle-aged, and Old.

By following this complete pipeline, the project evaluates different algorithms and also highlights the role of preprocessing, feature representation, and clustering in uncovering hidden patterns within complex datasets.

Dataset Description:

1. The dataset contains 21 columns. Since we are predicting "Age", we can say that there are 20 features: 9 Numerical features and 11 categorical features.

2. Our problem is a regression problem, since "Age" is a continuous numerical value here. In regression, model learns to predict exact numerical values like 20,24,30,55,72. On the other hand, if the age was classified as young, middle-age, old, we need to use classification. Now, we got the "Age" as a numerical value, so we are treating it as a regression task.

3. There are 41,118 data points to train our model. (41,118 rows)

4. There are 9 quantitative features and 11 categorical features.

5. The correlation heatmap shows that most features have only a weak connection to age. Among them, marital status stands out with the strongest negative relationship: younger people are more often single, while older individuals are more likely to be married or divorced. Education also shows a slight negative correlation, suggesting that younger clients generally have higher education levels than older ones. In contrast, features such as housing, loan, job, and campaign appear to have little to no direct link with age. The heatmap further highlights that several economic indicators, including employment variation rate and number of employees, are strongly correlated with one another, which points to redundancy in the dataset. Overall, the results indicate that categorical features like marital status and education provide more useful signals for predicting age than most of the banking or campaign-related numerical variables.

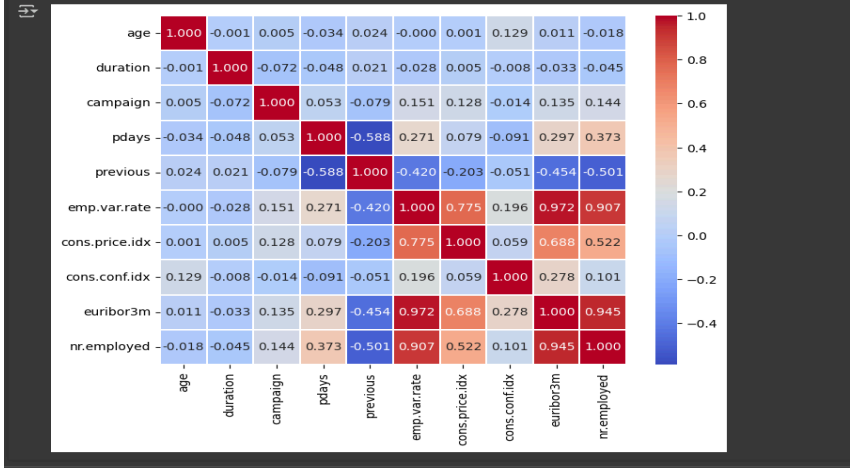
```
#Correlation matrix of whole dataset
correlation_matrix = numerical_data.corr()
correlation_matrix

```

	age	duration	campaign	pdays	previous	emp.var.rate	cons.price.idx	cons.conf.idx	euribor3m	nr.employed
age	1.000000	-0.000866	0.004594	-0.034369	0.024365	-0.000371	0.000857	0.129372	0.010767	-0.017725
duration	-0.000866	1.000000	-0.071699	-0.047577	0.020640	-0.027968	0.005312	-0.008173	-0.032897	-0.044703
campaign	0.004594	-0.071699	1.000000	0.052584	-0.079141	0.150754	0.127836	-0.013733	0.135133	0.144095
pdays	-0.034369	-0.047577	0.052584	1.000000	-0.587514	0.271004	0.078889	-0.091342	0.296899	0.372605
previous	0.024365	0.020640	-0.079141	-0.587514	1.000000	-0.420489	-0.203130	-0.050936	-0.454494	-0.501333
emp.var.rate	-0.000371	-0.027968	0.150754	0.271004	-0.420489	1.000000	0.775334	0.196041	0.972245	0.906970
cons.price.idx	0.000857	0.005312	0.127836	0.078889	-0.203130	0.775334	1.000000	0.058986	0.688230	0.522034
cons.conf.idx	0.129372	-0.008173	-0.013733	-0.091342	-0.050936	0.196041	0.058986	1.000000	0.277686	0.100513
euribor3m	0.010767	-0.032897	0.135133	0.296899	-0.454494	0.972245	0.688230	0.277686	1.000000	0.945154
nr.employed	-0.017725	-0.044703	0.144095	0.372605	-0.501333	0.906970	0.522034	0.100513	0.945154	1.000000

Correlation Matrix: Quantitative Features

```
# Plotting the heatmap for correlation matrix
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.3f', linewidths=0.3)
plt.show()
```



Heatmap: Quantitative Features

```python
from sklearn.preprocessing import LabelEncoder
dataset_encoded = dataset.copy()
encoder = LabelEncoder()

for col in dataset_encoded.select_dtypes(include=['object']).columns:
    dataset_encoded[col] = encoder.fit_transform(dataset_encoded[col])

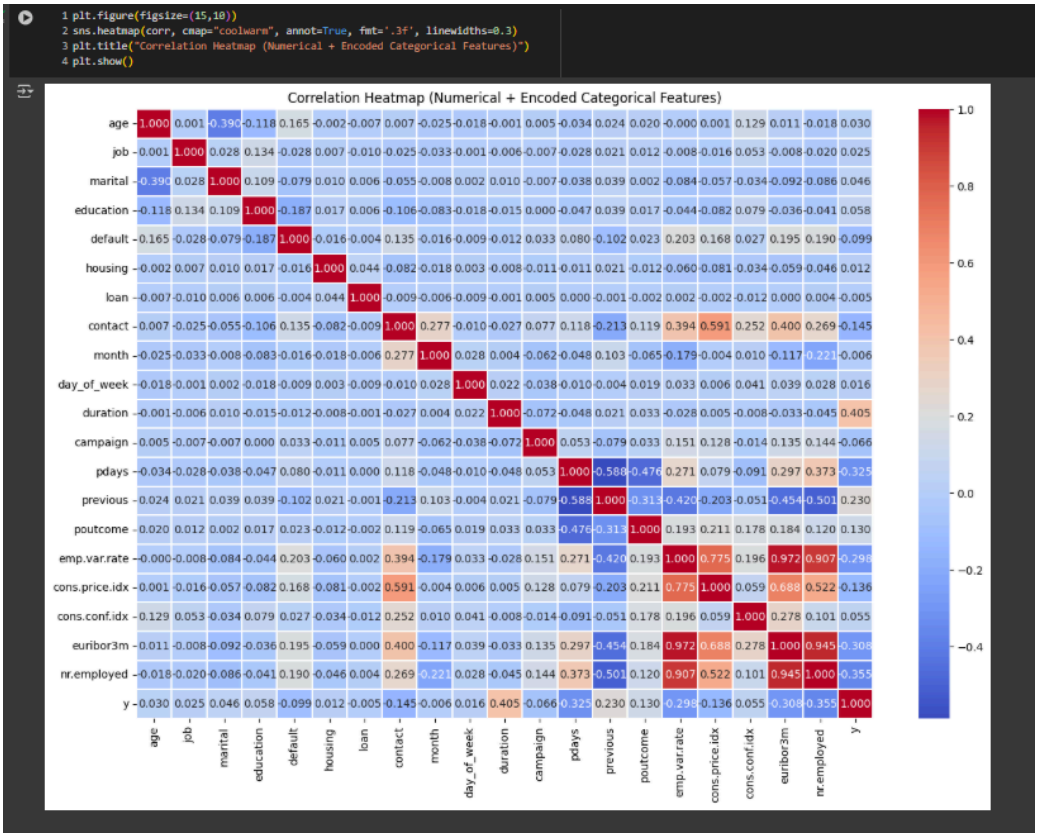
# Correlation matrix
corr = dataset_encoded.corr()
corr
```

	age	job	marital	education	default	housing	loan	contact	month	day_of_week	...	campaign	pdays	previous	poutcome	emp.var.rate	cons.price.idx	cons.conf.idx	euribor3m	nr.employed	y
age	1.000000	0.001250	-0.389753	-0.117892	0.164965	-0.001603	-0.007368	0.007021	-0.024877	-0.017572	...	0.004594	-0.034369	0.024365	0.019750	-0.000371	0.000857	0.129372	0.010767	-0.017725	0.030399
job	0.001250	1.000000	0.027897	0.134121	-0.028277	0.006962	-0.010209	-0.025132	-0.033213	-0.000844	...	-0.006923	-0.029468	0.020965	0.011504	-0.008271	-0.016017	0.052760	-0.007880	-0.019574	0.025122
marital	-0.389753	0.027897	1.000000	0.109220	-0.079450	0.010467	0.005788	-0.054501	-0.007629	0.002202	...	-0.007240	-0.037942	0.038889	0.001912	-0.084210	-0.057477	-0.033783	-0.091939	-0.086199	0.046203
education	-0.117892	0.134121	0.109220	1.000000	-0.186859	0.016825	0.006384	-0.105726	-0.082684	-0.017986	...	0.000371	-0.046626	0.038831	0.017009	-0.043778	-0.081607	0.078799	-0.036380	-0.041492	0.057799
default	0.164965	-0.028277	-0.079450	-0.186859	1.000000	-0.015815	-0.003782	0.135238	-0.015830	-0.008701	...	0.032825	0.080062	-0.102416	0.023417	0.203263	0.168073	0.026522	0.195336	0.189845	-0.099352
housing	-0.001603	0.006962	0.010467	0.016825	-0.015815	1.000000	0.044296	-0.082186	-0.018141	0.003339	...	-0.011010	-0.010551	0.021314	-0.011783	-0.060196	-0.080504	-0.033845	-0.059277	-0.045882	0.011552
loan	-0.007368	-0.010209	0.005788	0.006384	-0.003782	0.044296	1.000000	-0.008556	-0.005705	-0.009344	...	0.005166	0.000345	-0.001327	-0.001511	0.001849	-0.002430	-0.012025	0.000125	0.003903	-0.004909
contact	0.007021	-0.025132	-0.054501	-0.105726	0.135238	-0.082186	-0.008556	1.000000	0.276565	-0.009575	...	0.077368	0.117970	-0.212848	0.118744	0.393584	0.501474	0.251614	0.399773	0.269155	-0.144773
month	-0.024877	-0.033213	-0.007629	-0.082684	-0.015830	-0.018141	-0.005705	0.276565	1.000000	0.027677	...	-0.062059	-0.047891	0.103157	-0.065012	-0.178782	-0.004239	0.009652	-0.117264	-0.221425	-0.006065
day_of_week	-0.017572	-0.000844	0.002202	-0.017986	-0.008701	0.003339	-0.009344	-0.009575	0.027677	1.000000	...	-0.038288	-0.009531	-0.004102	0.018732	0.033245	0.005644	0.041465	0.039043	0.028380	0.015967
duration	-0.000866	-0.006490	0.010290	-0.015102	-0.011794	-0.007858	-0.000916	-0.026657	0.003690	0.021950	...	-0.071699	-0.047577	0.020640	0.033360	-0.027968	0.005312	-0.008173	-0.032897	-0.044703	0.405274
campaign	0.004594	-0.006923	-0.007240	0.000371	0.032825	-0.011010	0.005166	0.077368	-0.062059	-0.038288	...	1.000000	0.052584	-0.079141	0.032586	0.150754	0.127836	-0.013733	0.135133	0.144095	-0.066357
pdays	-0.034369	-0.029468	-0.037942	-0.046626	0.080062	-0.010551	0.000345	0.117970	-0.047891	-0.009531	...	0.052584	1.000000	-0.587514	-0.475619	0.271004	0.078889	-0.091342	0.296899	0.372605	-0.324914
previous	0.024365	0.020965	0.038889	0.038831	-0.102416	0.021314	-0.001327	-0.212848	0.103157	-0.004102	...	-0.079141	-0.587514	1.000000	-0.313110	-0.420489	-0.203130	-0.050936	-0.454494	-0.501333	0.230181
poutcome	0.019750	0.011504	0.001912	0.017009	0.023417	-0.011783	-0.001511	0.118744	-0.065012	0.018732	...	0.032586	-0.475619	-0.313110	1.000000	0.192972	0.211330	0.178289	0.184144	0.119689	0.129789
emp.var.rate	-0.000371	-0.008271	-0.084210	-0.043778	0.203263	-0.060196	0.001849	0.393584	-0.178782	0.033245	...	0.150754	0.271004	-0.420489	0.192972	1.000000	0.775334	0.196041	0.972245	0.906970	-0.298334
cons.price.idx	0.000857	-0.016017	-0.057477	-0.081607	0.168073	-0.080504	-0.002430	0.591474	-0.004239	0.005644	...	0.127836	0.078889	-0.203130	0.211330	0.775334	1.000000	0.058986	0.688230	0.522034	-0.136211
cons.conf.idx	0.129372	0.052760	-0.033783	0.078799	0.026522	-0.033845	-0.012025	0.251614	0.009652	0.041465	...	-0.013733	-0.091342	-0.050936	0.178289	0.196041	0.058986	1.000000	0.277686	0.100513	0.054878
euribor3m	0.010767	-0.007880	-0.091939	-0.036380	0.195336	-0.059277	0.000125	0.399773	-0.117264	0.039043	...	0.135133	0.296899	-0.454494	0.184144	0.972245	0.688230	0.277686	1.000000	0.945154	-0.307771
nr.employed	-0.017725	-0.019574	-0.086199	-0.041492	0.189845	-0.045882	0.003903	0.269155	-0.221425	0.028380	...	0.144095	0.372605	-0.501333	0.119689	0.906970	0.522034	0.100513	0.945154	1.000000	-0.354678
y	0.030399	0.025122	0.046203	0.057799	-0.099352	0.011552	-0.004909	-0.144773	-0.006065	0.015967	...	-0.066357	-0.324914	0.230181	0.129789	-0.298334	-0.136211	0.054878	-0.307771	-0.354678	1.000000

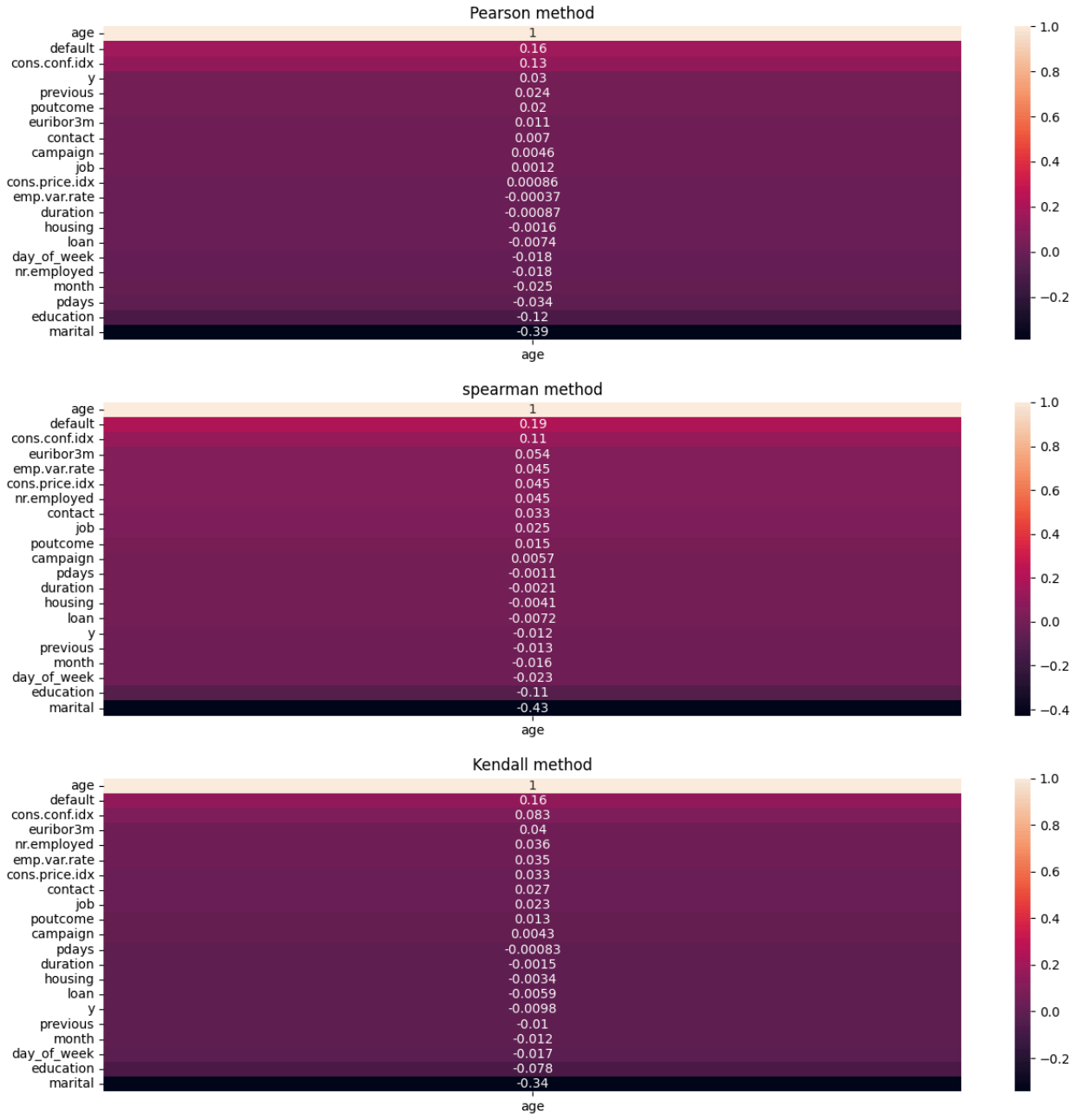
21 rows × 21 columns

Correlation Matrix: All features

```python
plt.figure(figsize=(15,10))
sns.heatmap(corr, cmap="coolwarm", annot=True, fmt='.3f', linewidths=0.3)
plt.title("Correlation Heatmap (Numerical + Encoded Categorical Features)")
plt.show()
```



Heatmap: All features



Correlation plot between features and target in 3 different methods(Bottom one has the minimum ratio of relation)

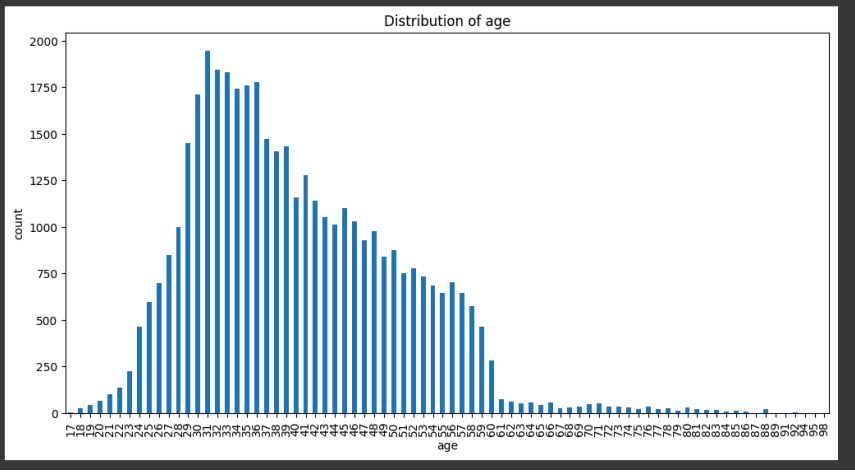
Imbalanced Dataset:

There are 78 unique classes of age. [N=78]

```
[50]  1 unique_counts=dataset_encoded.nunique()
      2 print(unique_counts)
```

```
age               78
job               12
marital            4
education          8
default            3
housing            3
loan               3
contact            2
month             10
day_of_week        5
duration        1544
campaign          42
pdays             27
previous           8
poutcome           3
emp.var.rate      10
cons.price.idx    26
cons.conf.idx     26
euribor3m        316
nr.employed       11
y                  2
dtype: int64
```

```
plt.figure(figsize=(12,6))
plt.title(f'Distribution of age')
numerical_data['age'].value_counts().sort_index().plot(kind='bar', rot=90, xlabel="age",ylabel='count')
# plt.xticks(ticks=range(0, len(ax.get_xticks()), 5),
#            labels=[lab.get_text() for i, lab in enumerate(ax.get_xticklabels()) if i % 5 == 0])
plt.show()
```



The bar plot shows the counts of people of different ages. So, all unique classes do not have an equal number of instances.

EDA:

At first, we did descriptive analysis for both quantitative features and categorical features.

```
[86]  1 numerical_data.describe().T
```

	count	mean	std	min	25%	50%	75%	max
age	41188.0	40.024060	10.421250	17.000	32.000	38.000	47.000	98.000
duration	41188.0	258.285010	259.279249	0.000	102.000	180.000	319.000	4918.000
campaign	41188.0	2.567593	2.770014	1.000	1.000	2.000	3.000	56.000
pdays	41188.0	962.475454	186.910907	0.000	999.000	999.000	999.000	999.000
previous	41188.0	0.172963	0.494901	0.000	0.000	0.000	0.000	7.000
emp.var.rate	41188.0	0.081886	1.570960	-3.400	-1.800	1.100	1.400	1.400
cons.price.idx	41188.0	93.575664	0.578840	92.201	93.075	93.749	93.994	94.767
cons.conf.idx	41188.0	-40.502600	4.628198	-50.800	-42.700	-41.800	-36.400	-26.900
euribor3m	41188.0	3.621291	1.734447	0.634	1.344	4.857	4.961	5.045
nr.employed	41188.0	5167.035911	72.251528	4963.600	5099.100	5191.000	5228.100	5228.100

Summary satistics of Categorical features

```
[53]  1 categorical_data.describe().T
```

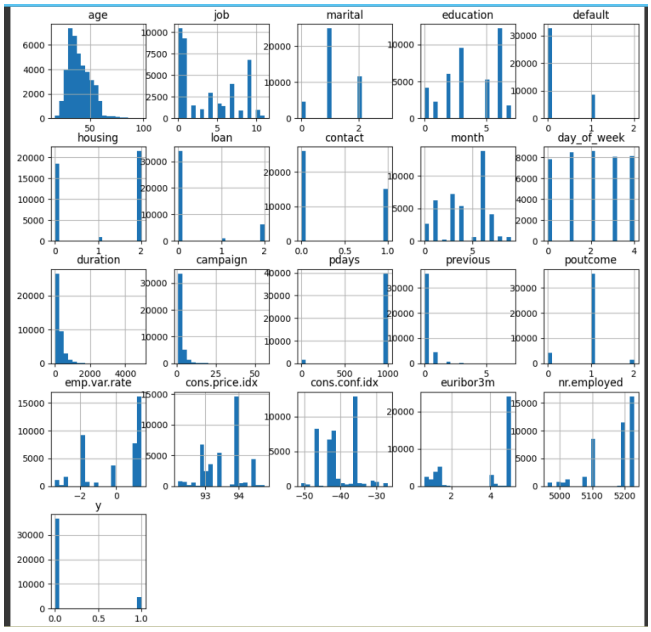
	count	unique	top	freq
job	41188	12	admin.	10422
marital	41188	4	married	24928
education	41188	8	university.degree	12168
default	41188	3	no	32588
housing	41188	3	yes	21576
loan	41188	3	no	33950
contact	41188	2	cellular	26144
month	41188	10	may	13769
day_of_week	41188	5	thu	8623
poutcome	41188	3	nonexistent	35563
y	41188	2	no	36548

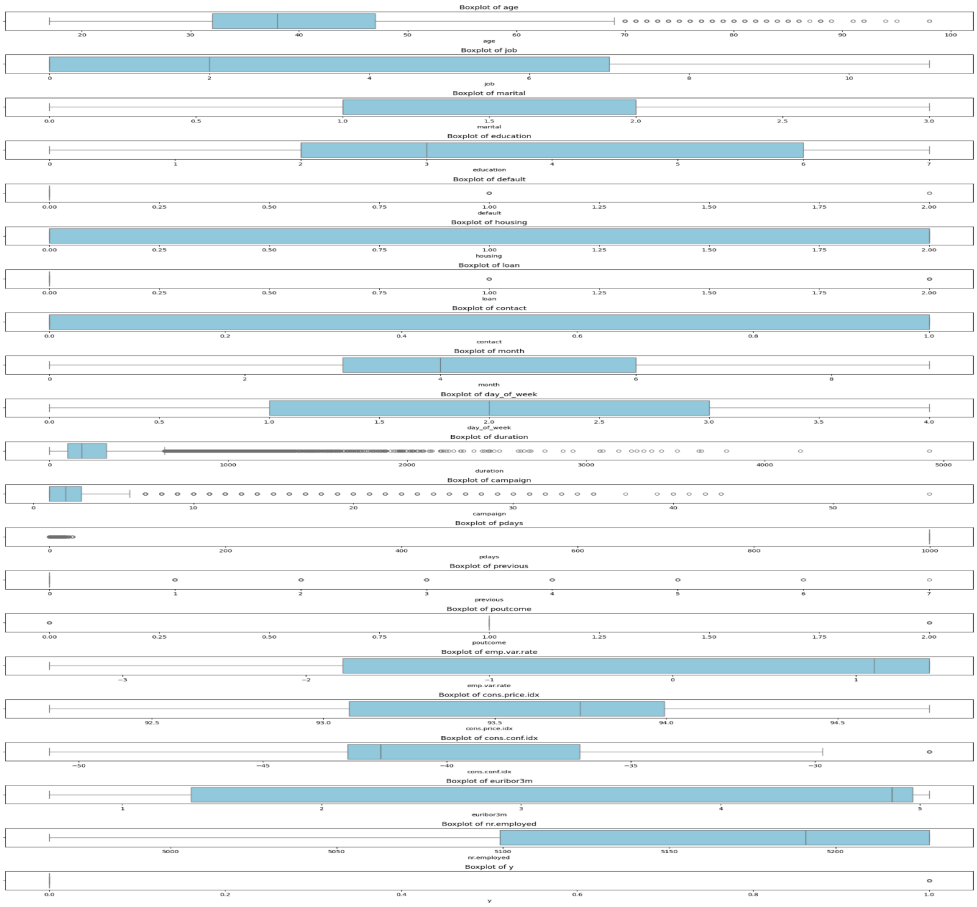
Secondly, we calculated the skewness of the features, skewness helps us to understand the symmetry of feature distributions.

Skew in features

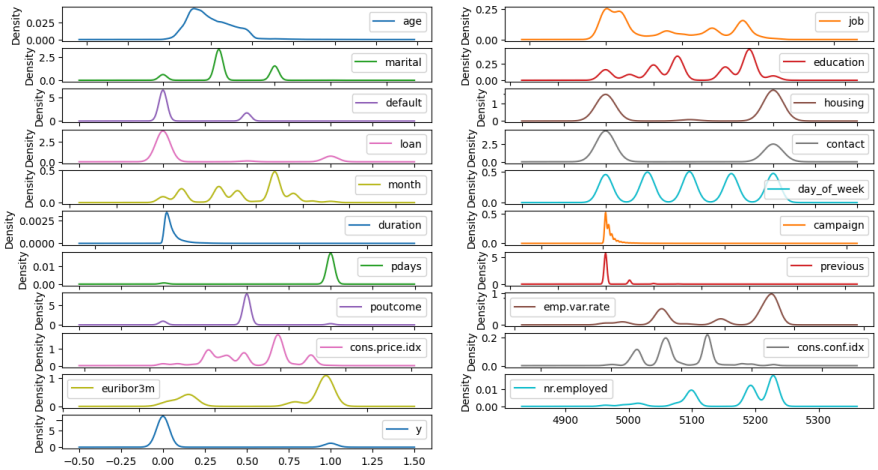
```
1 dataset_encoded.skew()
```

	0
age	0.784697
job	0.450818
marital	-0.057136
education	-0.235148
default	1.435659
housing	-0.143783
loan	1.816348
contact	0.559720
month	-0.309755
day_of_week	0.008172
duration	3.263141
campaign	4.762507
pdays	-4.922190
previous	3.832042
poutcome	-0.877789
emp.var.rate	-0.724096
cons.price.idx	-0.230888
cons.conf.idx	0.303180
euribor3m	-0.709188





Density plot of features



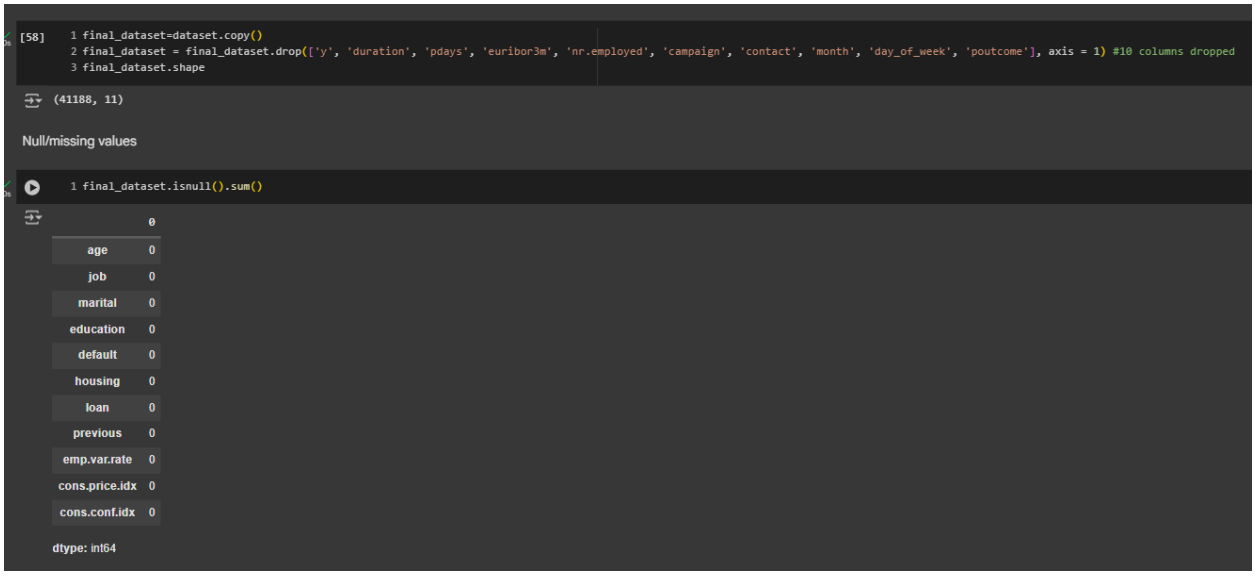
Data Pre-Processing:

A careful preprocessing strategy was applied to refine the dataset and remove redundant features. The original target variable y (subscription outcome) was excluded, as it is irrelevant to the chosen prediction task of estimating age. Similarly, duration showed near-zero correlation with age and was removed.

Several variables displayed high multicollinearity. In particular, euribor3m and nr.employed were strongly correlated with emp.var.rate. To reduce redundancy, only emp.var.rate was retained as the representative feature. The pdays variable was also dropped, as it was dominated by the sentinel value 999, producing extreme skewness (≈ –4.9) and largely overlapping in information with previous. In this case, previous was preserved as the more informative variable.

Other campaign-related attributes, such as campaign, contact, month, day_of_week, and poutcome, exhibited near-zero correlation with age and were removed to simplify the model. By contrast, categorical features such as job, marital status, and education showed the strongest relationships with age, with marital status demonstrating the largest absolute correlation. Binary categorical variables (default, housing, and loan) were also retained, as they could contribute useful predictive information.

In summary, the preprocessing process resulted in a linear dataset that emphasized the most relevant variables: job, marital status, education, default, housing, loan, previous, emp.var.rate, cons.price.idx, and cons.conf.idx. This selection reduced noise, minimized redundancy, and ensured that the subsequent modeling process was built on features most likely to contribute meaningful predictive power.



```
[58]  1 final_dataset=dataset.copy()
      2 final_dataset = final_dataset.drop(['y', 'duration', 'pdays', 'euribor3m', 'nr.employed', 'campaign', 'contact', 'month', 'day_of_week', 'poutcome'], axis = 1) #10 columns dropped
      3 final_dataset.shape

(41188, 11)
```

Null/missing values

```
1 final_dataset.isnull().sum()
```

	0
age	0
job	0
marital	0
education	0
default	0
housing	0
loan	0
previous	0
emp.var.rate	0
cons.price.idx	0
cons.conf.idx	0

dtype: int64

```
[62]  1 print(final_dataset['job'].unique())
      2 print(final_dataset['marital'].unique())
      3 print(final_dataset['education'].unique())

['housemaid' 'services' 'admin.' 'blue-collar' 'technician' 'retired'
 'management' 'unemployed' 'self-employed' 'unknown' 'entrepreneur'
 'student']
['married' 'single' 'divorced' 'unknown']
['basic.4y' 'high.school' 'basic.6y' 'basic.9y' 'professional.course'
 'unknown' 'university.degree' 'illiterate']
```

```
[63]  1 from sklearn.preprocessing import OneHotEncoder
      2
      3
      4 categorical_columns = final_dataset.select_dtypes(include=['object']).columns.tolist()
      5 encoder = OneHotEncoder(sparse_output=False)
      6
      7 one_hot_encoded = encoder.fit_transform(final_dataset[categorical_columns])
      8
      9 one_hot_df = pd.DataFrame(one_hot_encoded, columns=encoder.get_feature_names_out(categorical_columns))
     10
     11 processed_dataset = pd.concat([final_dataset, one_hot_df], axis=1)
     12
     13 processed_dataset= processed_dataset.drop(categorical_columns, axis=1)
     14 print(f"Encoded Employee data : \n{processed_dataset}")

Encoded Employee data :
       age  default  housing  loan  previous  emp.var.rate  cons.price.idx  \
0       56        0        0     0         0           1.1          93.994
1       57        1        0     0         0           1.1          93.994
2       37        0        2     0         0           1.1          93.994
3       40        0        0     0         0           1.1          93.994
4       56        0        0     2         0           1.1          93.994
...    ...      ...      ...   ...       ...           ...             ...
41183   73        0        2     0         0          -1.1          94.767
41184   46        0        0     0         0          -1.1          94.767
41185   56        0        2     0         0          -1.1          94.767
41186   44        0        0     0         0          -1.1          94.767
41187   74        0        2     0         1          -1.1          94.767

       cons.conf.idx  job_admin.  job_blue-collar  marital_single  \
```

Encoding: Using One-Hot Encoding for job, marital, and education.

```
applying log1p on previous

[64]  1 # Suppose your dataframe is called df
      2 print("Skewness before:", processed_dataset['previous'].skew())
      3 processed_dataset['previous_log'] = np.log1p(processed_dataset['previous'])
      4
      5 # Drop the old 'previous' column if you don't need it anymore
      6 processed_dataset = processed_dataset.drop('previous', axis=1)
      7
      8 # Check skewness after transformation
      9 print("Skewness after:", processed_dataset['previous_log'].skew())
     10 processed_dataset

Skewness before: 3.8320422428611836
Skewness after: 2.587040192505977
```

	age	default	housing	loan	emp.var.rate	cons.price.idx	cons.conf.idx	job_admin.	job_blue-collar	job_entrepreneur	...	marital_unknown	education_basic.4y	education_basic.6y	education_basic.9y	education_high.school	education_illiterate	edu
0	56	0	0	0	1.1	93.994	-36.4	0.0	0.0	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0	
1	57	1	0	0	1.1	93.994	-36.4	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	
2	37	0	2	0	1.1	93.994	-36.4	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	
3	40	0	0	0	1.1	93.994	-36.4	1.0	0.0	0.0	...	0.0	0.0	1.0	0.0	0.0	0.0	
4	56	0	0	2	1.1	93.994	-36.4	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
41183	73	0	2	0	-1.1	94.767	-50.8	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	
41184	46	0	0	0	-1.1	94.767	-50.8	0.0	1.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	
41185	56	0	2	0	-1.1	94.767	-50.8	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	
41186	44	0	0	0	-1.1	94.767	-50.8	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	
41187	74	0	2	0	-1.1	94.767	-50.8	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	

41188 rows × 32 columns

Scaling: We can see that the previous feature was heavily skewed. log1p transformation compresses the large values in order to make the distribution more balanced. Our numeric features (previous_log, emp.var.rate, cons.price.idx, cons.conf.idx) are on different scales and contain outliers.

Robust scaling

```
[65]  1 from sklearn.preprocessing import RobustScaler
      2 numeric_features = ['previous_log', 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx']
      3
      4 # Initialize RobustScaler
      5 scaler = RobustScaler()
      6
      7 # Fit and transform
      8 processed_dataset[numeric_features] = scaler.fit_transform(processed_dataset[numeric_features])
      9
     10 # Check result
     11 print(processed_dataset[numeric_features].head())
```

```
   previous_log  emp.var.rate  cons.price.idx  cons.conf.idx
0           0.0           0.0        0.266594       0.857143
1           0.0           0.0        0.266594       0.857143
2           0.0           0.0        0.266594       0.857143
3           0.0           0.0        0.266594       0.857143
4           0.0           0.0        0.266594       0.857143
```

And now after log1p+Rebust scaling helps regression models learn patterns in age prediction more effectively.

Dataset Splitting:

Dataset splitting

```
[67]  1 from sklearn.model_selection import train_test_split
      2
      3
      4 # Separate features  and target
      5 data = processed_dataset.drop('age', axis=1)   # all features
      6 target = processed_dataset['age']               # target variable
      7
      8 # Random split 70-30
      9 X_train, X_test, y_train, y_test = train_test_split(data, target, test_size = 0.3, random_state=000, shuffle=True   )
     10 print("X_train shape:", X_train.shape)
     11 print("y_train shape:", y_train.shape)
     12 print("X_test shape:", X_test.shape)
     13 print("y_test shape:", y_test.shape)
```

```
X_train shape: (28831, 31)
y_train shape: (28831,)
X_test shape: (12357, 31)
y_test shape: (12357,)
```

Since the target variable in this project is age(continuous numeric feature) the task is a regression problem. Stratified splitting is typically applied in classification tasks to maintain the proportion of classes across training and testing sets. In regression, this constraint is unnecessary, and a standard random split is sufficient to ensure a representative division of the data. Therefore, the dataset was divided into training and testing subsets using a random split strategy.

Model Training & Testing:

Supervised:

```
[69]  1 # 1. Linear Regression
      2
      3 lin_reg = LinearRegression()
      4 lin_reg.fit(X_train, y_train)
      5 y_pred_lr = lin_reg.predict(X_test)
      6 print("Test set predictions:\n", y_pred_lr)
      7 print("The Training accuracy of the model is {:.2f}".format(lin_reg.score(X_train, y_train)))
      8 print("The Testing accuracy of the model is {:.2f}".format(lin_reg.score(X_test, y_test)))

Test set predictions:
 [37.92242277 42.46605834 40.78878229 ... 39.05485404 42.7600231
 39.85017702]
The Training accuracy of the model is 0.40
The Testing accuracy of the model is 0.40
```

Linear Regression: For predicting continuous outcomes, Linear Regression is one of the most fundamental supervised learning algorithms. It operates by fitting a straight line (or hyperplane in higher dimensions) that best represents the relationship between input features and the target variable, minimizing the error between predicted and actual values. In this project, the model was applied to capture and estimate the linear relationship between the selected features and age.

```
[70]  1 # 2. Decision Tree
      2
      3 dt_reg = DecisionTreeRegressor(random_state=000, max_depth=10)
      4 dt_reg.fit(X_train, y_train)
      5 y_pred_dt = dt_reg.predict(X_test)
      6 print("Test set predictions:\n", y_pred_dt)
      7 print("The Training accuracy of the model is {:.2f}".format(dt_reg.score(X_train, y_train)))
      8 print("The Testing accuracy of the model is {:.2f}".format(dt_reg.score(X_test, y_test)))

Test set predictions:
 [38.53388042 41.3253012  41.36807818 ... 37.60033306 38.53388042
 42.33922652]
The Training accuracy of the model is 0.47
The Testing accuracy of the model is 0.42
```

Decision Letter: Decision Tree is a supervised learning algorithm that partitions the dataset into progressively smaller subsets according to feature values, forming a structure similar to a flowchart of decisions. Each split is chosen to maximize the separation of the target variable, creating branches that lead to more homogeneous groups. In this project, the tree was constrained to a maximum depth of 10, balancing model complexity with the risk of overfitting.

```python
# 3. Neural Network (MLP)

nn_reg = MLPRegressor(hidden_layer_sizes=(64,32), max_iter=500, random_state=000)
nn_reg.fit(X_train, y_train)
y_pred_nn = nn_reg.predict(X_test)
print("Test set predictions:\n", y_pred_nn)
print("The Training accuracy of the model is {:.2f}".format(nn_reg.score(X_train, y_train)))
print("The Testing accuracy of the model is {:.2f}".format(nn_reg.score(X_test, y_test)))
```

```
Test set predictions:
 [36.18117409 43.28931078 41.67067357 ... 43.54599886 41.30094249
 45.46273021]
The Training accuracy of the model is 0.49
The Testing accuracy of the model is 0.44
```

Neural Network: By including a neural network in the system, the project not only demonstrates the application of modern deep learning techniques but also provides a meaningful comparison against classical regression methods. This comparison highlights whether the added complexity of neural networks leads to measurable improvements in predicting age, thereby offering deeper insight into the trade-offs between interpretability and predictive power.

Unsupervised:

```python
#unsupervised Learning
#implementing K means Clustering in this dataframe

def implement_KMC(df_temp,cluster_num):
  km = KMeans(n_clusters=cluster_num)  #defining the cluster number
  y_predicted = km.fit_predict(processed_dataset[['age']])
  return km,y_predicted #returns the model and the predicted assigned cluster to each entry
  #Running the method
cluster = 3
km,y_predicted = implement_KMC(processed_dataset,cluster)  ##stores the model and the predicted values
processed_dataset["Group_cluster"]=y_predicted
print(processed_dataset[['age','Group_cluster']].head(15))
```

	age	Group_cluster
0	56	2
1	57	2
2	37	1
3	40	0
4	56	2
5	45	0
6	59	2
7	41	0
8	24	1
9	25	1
10	41	0
11	25	1
12	29	1
13	57	2
14	35	1

```
[73]  1 label_map = {2: 'Old', 0: 'Young', 1: 'Middle-aged'}
      2 processed_dataset['Label'] = processed_dataset['Group_cluster'].map(label_map)
      3 print("Original Distribution of the Group\n")
      4 processed_dataset['Label'].value_counts()

Original Distribution of the Group
```

Label	count
Middle-aged	21175
Young	13587
Old	6426

dtype: int64

KMeans Clustering: To complement the supervised learning models, K-Means Clustering was applied as an unsupervised approach to group individuals based on their age. The algorithm was configured with three clusters (k=3), aiming to segment the population into meaningful categories: Young, Middle-aged, and Old. After clustering, labels were mapped to these categories for easier interpretation. The distribution revealed that the majority of clients fell into the Middle-aged group (21,175 individuals), followed by the Young group (13,587 individuals), and the smallest proportion in the Old group (6,426 individuals). This segmentation highlights the demographic composition of the dataset and provides an additional perspective beyond regression, demonstrating how clustering can uncover age-based group patterns that may be useful for marketing and strategic decision-making.

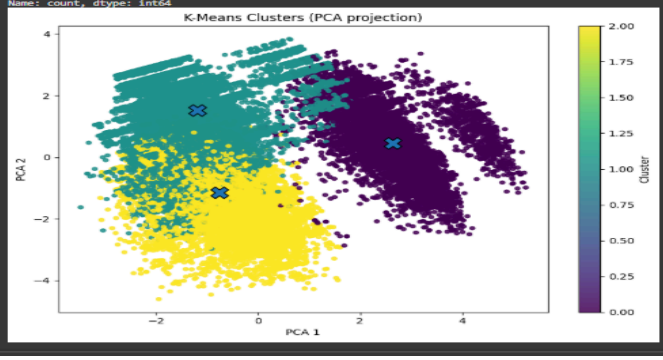
```
print("Cluster counts on clustered rows:")
print(processed_dataset.loc[idx, 'Group_cluster'].value_counts().sort_index())
if 'Label' in processed_dataset.columns:
    print("\nLabel counts:")
    print(processed_dataset.loc[idx, 'Label'].value_counts())

# 5) PCA -> 2D and plot
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)
centroids_pca = pca.transform(km.cluster_centers_)

plt.figure(figsize=(8,6))
scatter = plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_pred, cmap='viridis', s=18, alpha=0.85)
plt.scatter(centroids_pca[:, 0], centroids_pca[:, 1], marker='X', s=220, edgecolor='black', linewidth=1.0)
plt.title("K-Means Clusters (PCA projection)")
plt.xlabel("PCA 1")
plt.ylabel("PCA 2")
cbar = plt.colorbar(scatter)
cbar.set_label("Cluster")
plt.tight_layout()
plt.show()
```

```
Cluster counts on clustered rows:
Group_cluster
0    10586
1    11323
2    19279
Name: count, dtype: int64

Label counts:
Label
Middle-aged    19279
Old            11323
Young          10586
Name: count, dtype: int64
```



Comparison Analysis:

Comparison analysis

```python
# Evaluation Function

def evaluate_model(name, y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)


    r2 = r2_score(y_true, y_pred)
    return {
        'Model': name,

        'MSE/loss': round(mse,2),
        'R2 Score': round(r2,2)
    }

# Collect results
results = []
results.append(evaluate_model("Linear Regression", y_test, y_pred_lr))
results.append(evaluate_model("Decision Tree", y_test, y_pred_dt))
results.append(evaluate_model("Neural Network", y_test, y_pred_nn))

# Convert to DataFrame
results_df = pd.DataFrame(results)
print("\nModel Comparison:")
print(results_df)
```

```
Model Comparison:
                Model  MSE/loss  R2 Score
0   Linear Regression     64.90      0.40
1       Decision Tree     62.63      0.42
2      Neural Network     60.31      0.44
```

```python
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve
import numpy as np

def combined_loss_curves(lr, dt, nn, X_train, y_train):
    plt.figure(figsize=(10,6))

    # Linear Regression Learning Curve

    train_sizes, train_scores, val_scores = learning_curve(
        lr, X_train, y_train, cv=5, scoring='neg_mean_squared_error',
        train_sizes=np.linspace(0.1, 1.0, 10), n_jobs=-1
    )
    lr_val_loss = -np.mean(val_scores, axis=1)
    plt.plot(train_sizes, lr_val_loss, 'o-', label="Linear Regression (Val Loss)")

    # Decision Tree Learning Curve

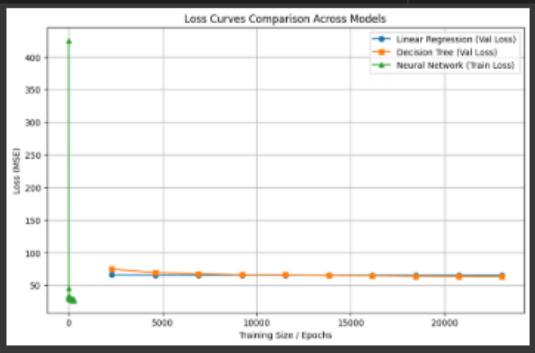
    train_sizes, train_scores, val_scores = learning_curve(
        dt, X_train, y_train, cv=5, scoring='neg_mean_squared_error',
        train_sizes=np.linspace(0.1, 1.0, 10), n_jobs=-1
    )
    dt_val_loss = -np.mean(val_scores, axis=1)
    plt.plot(train_sizes, dt_val_loss, 's-', label="Decision Tree (Val Loss)")


    # Neural Network Loss Curve

    plt.plot(range(1, len(nn.loss_curve_)+1), nn.loss_curve_, '^-', label="Neural Network (Train Loss)")


    # Formatting

    plt.title("Loss Curves Comparison Across Models")
    plt.xlabel("Training Size / Epochs")
    plt.ylabel("Loss (MSE)")
    plt.legend()
    plt.grid(True)
    plt.show()
combined_loss_curves(lin_reg, dt_reg, nn_reg, X_train, y_train)
```



The comparison of learning curves across models provides insight into their training behavior and performance. For **Linear Regression**, the curve remains almost flat with a slight downward trend, indicating that the model quickly reaches its optimal performance and does not benefit significantly. In contrast, the **Decision Tree** shows a curve that begins at a higher error level but decreases as the training size increases, eveentually stabilizing. This suggests that the tree improves its predictive ability with more data before reaching a plateau. The **Neural Network** starting with a high error that rapidly decreases over the course of training. This demonstrates the model's ability to learn complex patterns quickly and minimize the error efficiently. Among all models, the Neural Network shows the most rapid improvement, highlighting its efectiveness in capturing non-linear relationships within the dataset.

Conclusion:

This project focused on predicting a person's age from the Bank Marketing dataset using Linear Regression, Decision Tree Regressor, and a Neural Network, while also applying K-Means clustering to explore age-based groupings. The models produced varying levels of performance: the Neural Network demonstrated a stronger ability to capture non-linear patterns compared to Linear Regression and Decision Tree, yet all models faced challenges due to the weak correlations between most features and age. Attributes such as job, marital status, and education contributed some predictive power, but many banking and campaign-related variables, including housing and loan, showed little direct relationship with age.

A significant challenge in this project was managing categorical variables and skewed features, which required careful preprocessing steps such as encoding, scaling, and log transformation. These techniques were essential for preparing the data in a way that models could interpret effectively.

Overall, the project highlighted the strengths and limitations of different regression models and emphasized the critical role of data preprocessing in achieving meaningful results. It also showed that predicting age from this dataset is inherently difficult, but the process provided valuable insights into model behavior, feature importance, and the balance between model complexity and interpretability.